

Python Literacy

Python 리터러시

01 Python 소개

목차 01 Python 소개

1. Python 언어와 Python 생태계
2. Python을 이용한 개발 트렌드와 다양한 사용 사례

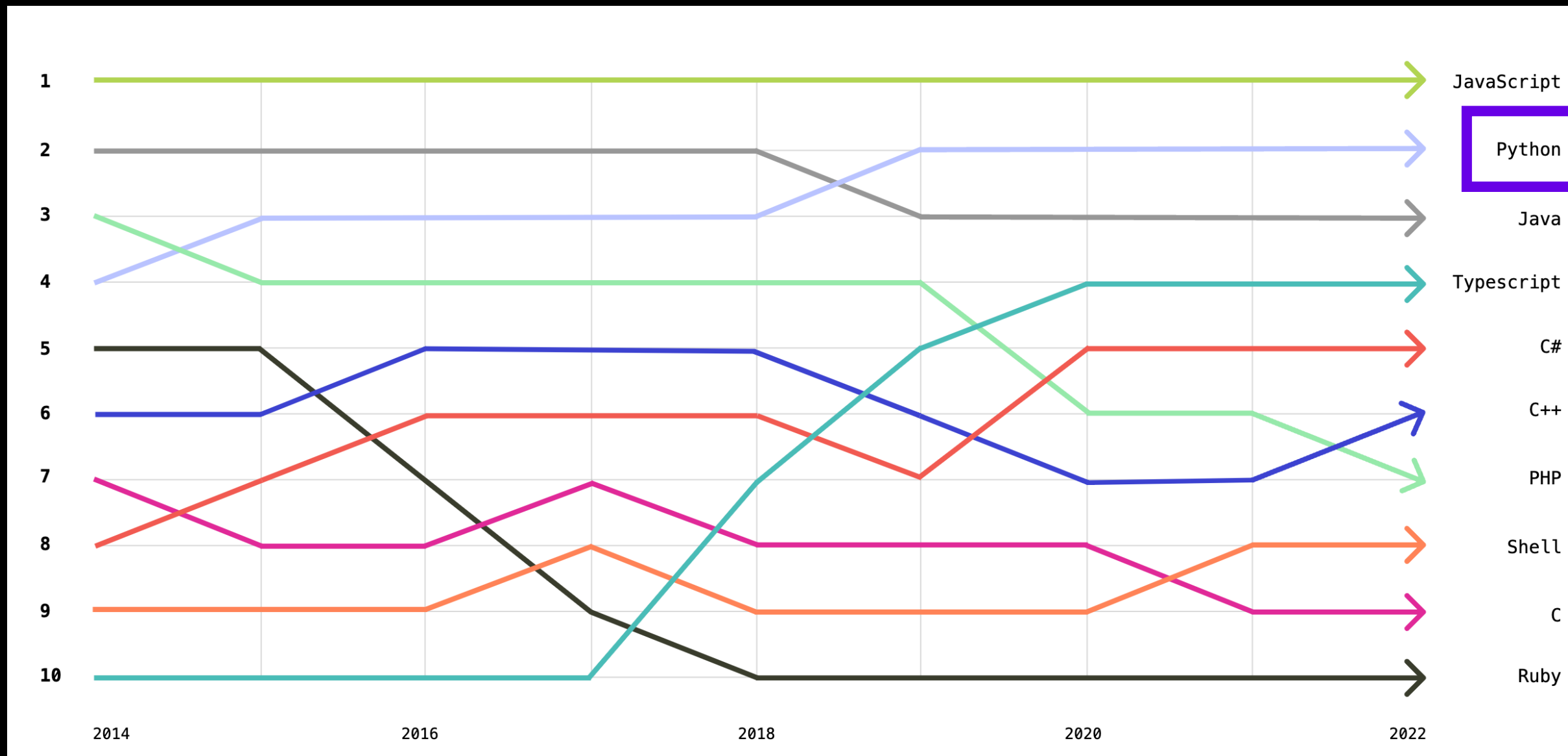
1. Python 언어와 Python 생태계

프로그래밍 언어 (Programming Languages)

Python 리터러시

01 Python 소개

1. Python 언어와
Python 생태계

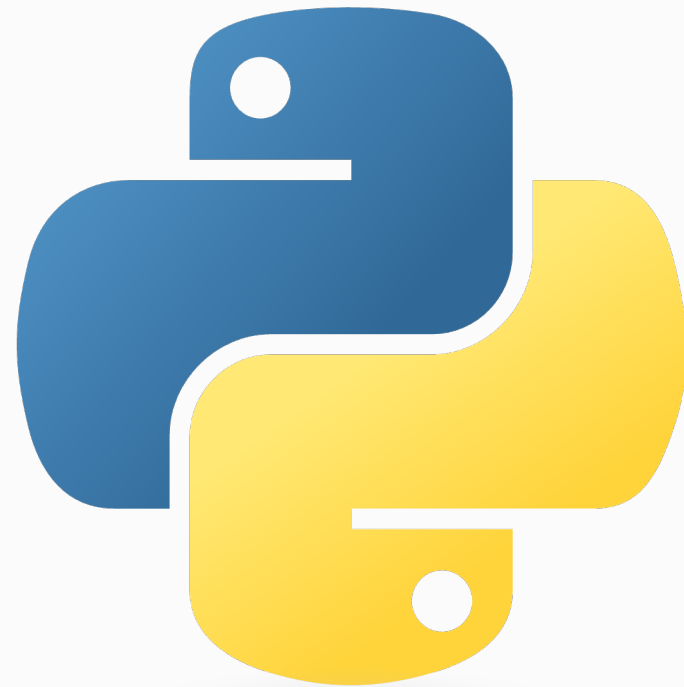


프로그래밍

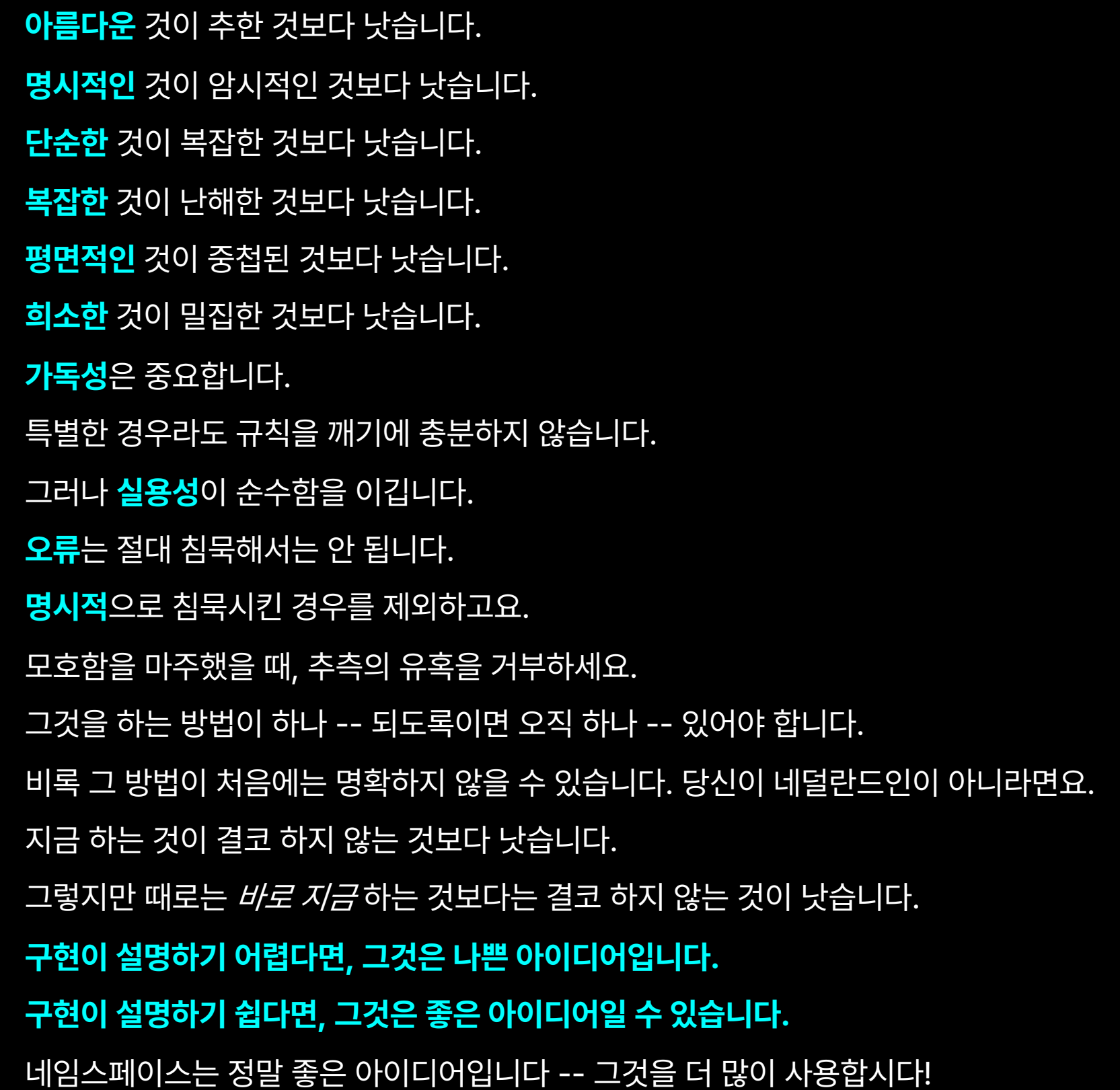
- 프로그램을 만드는 작업
- 컴퓨터에게 일을 시키는 방법

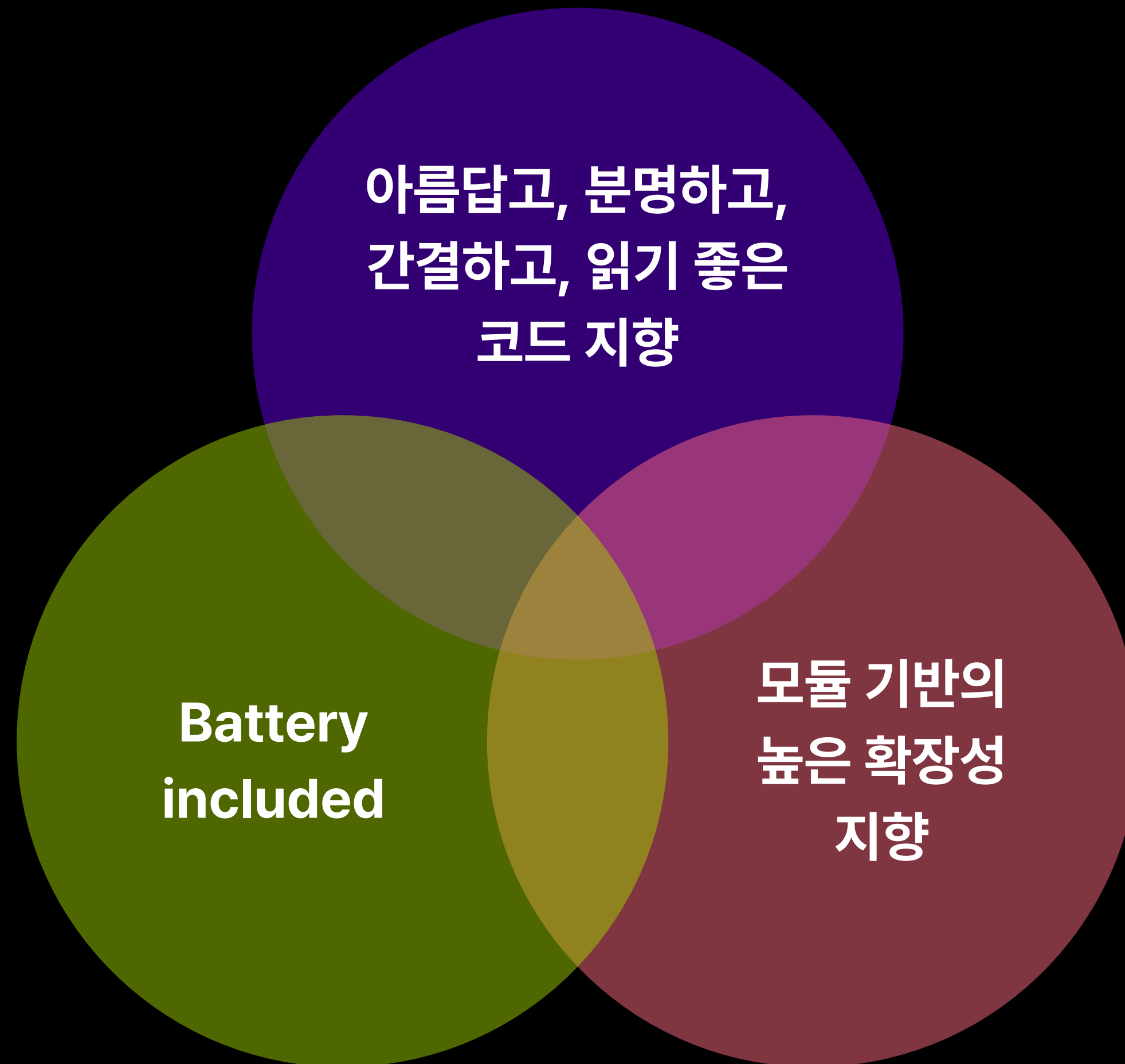
프로그래밍 언어

- 프로그램을 만들기 위해
컴퓨터와 통신하는 언어



Guido Van Rossum에 의해 개발됨
ABC 프로그래밍 언어의 후속으로 개발
1991년 첫 번째 릴리즈







```
#include <stdio.h>

int main()
{
    printf("Hi, Serendippers!");

    return 0;
}
```



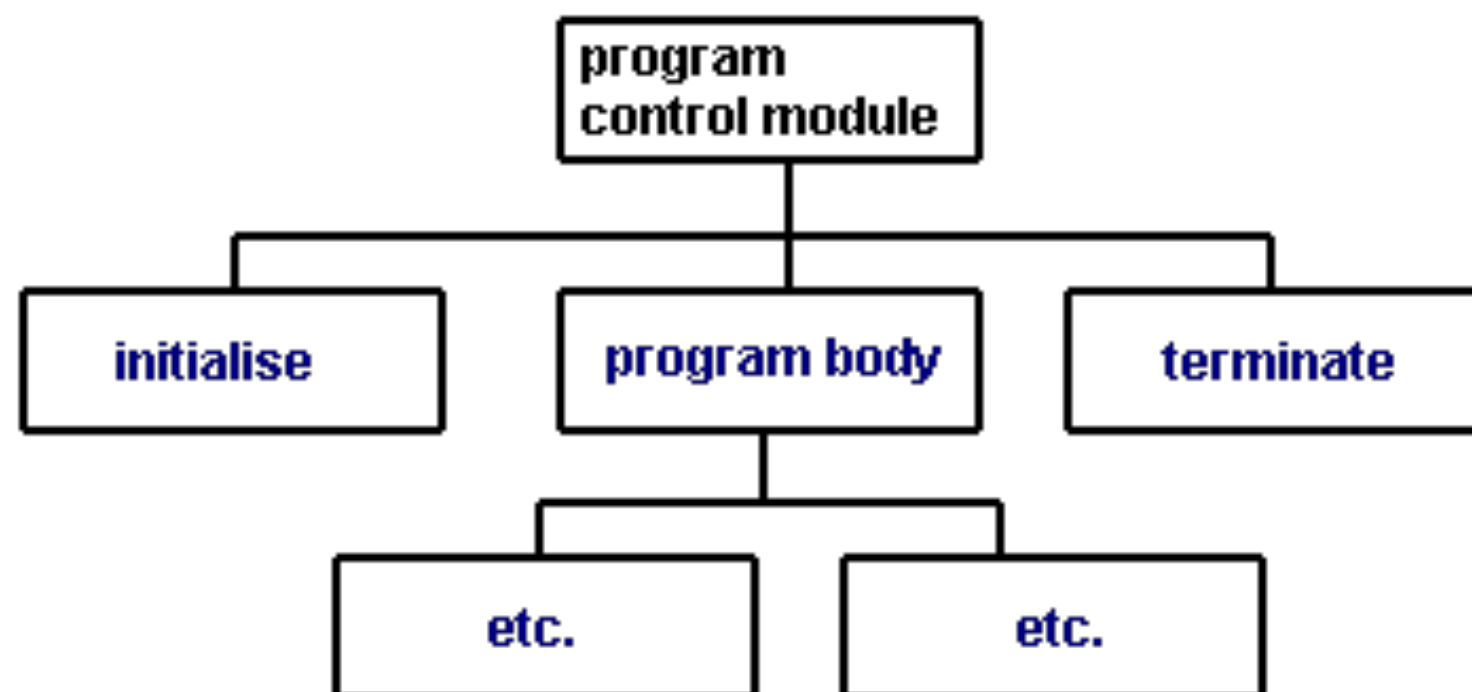
```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hi, Serendippers!");
    }
}
```



```
print("Hi, Serendippers!")
```



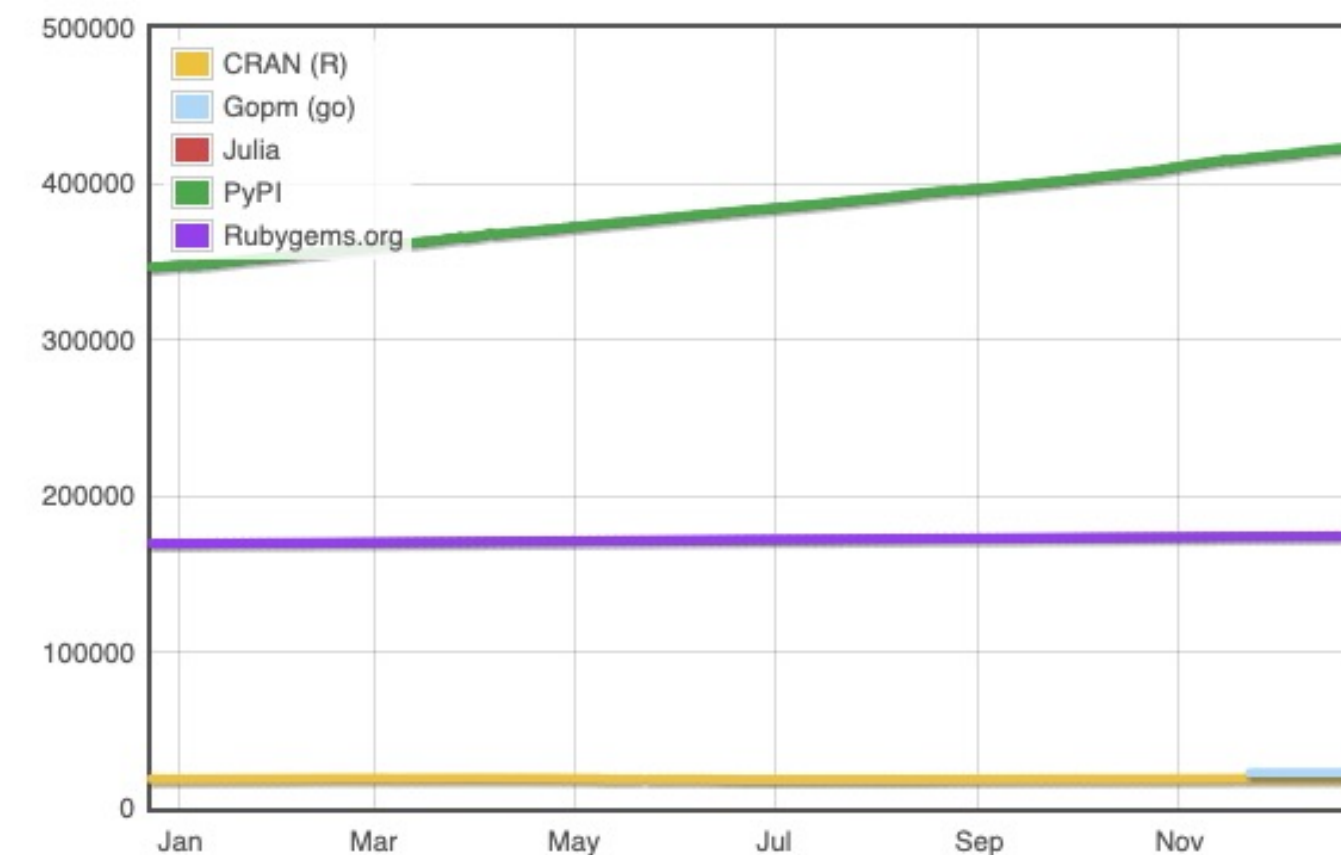
Hierarchy of modules



- **프로그래밍 모듈**

- 독립적으로 실행가능하며 교체가능한 프로그램
- 일반적으로 특정한 역할을 수행

Module Counts



- **모듈 인터페이스**

모듈과 메인 프로그램을 연결하는 규약

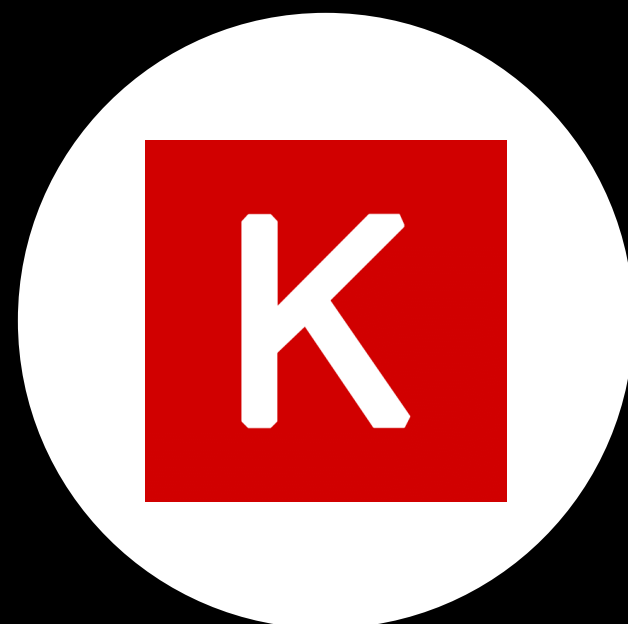
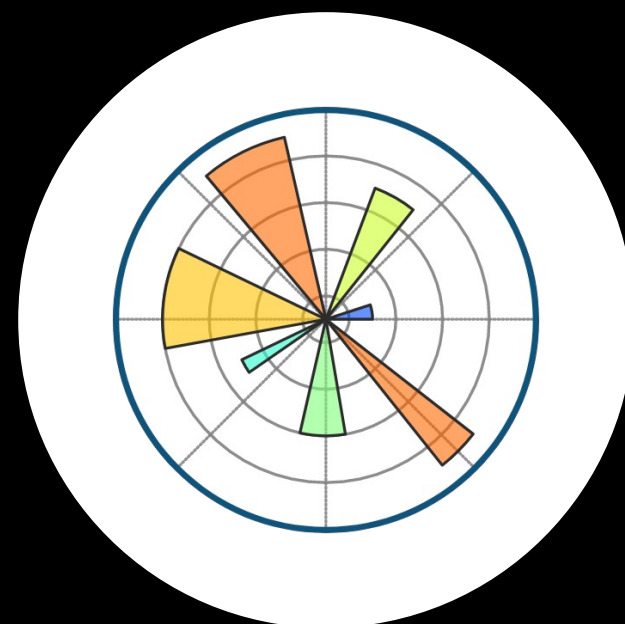
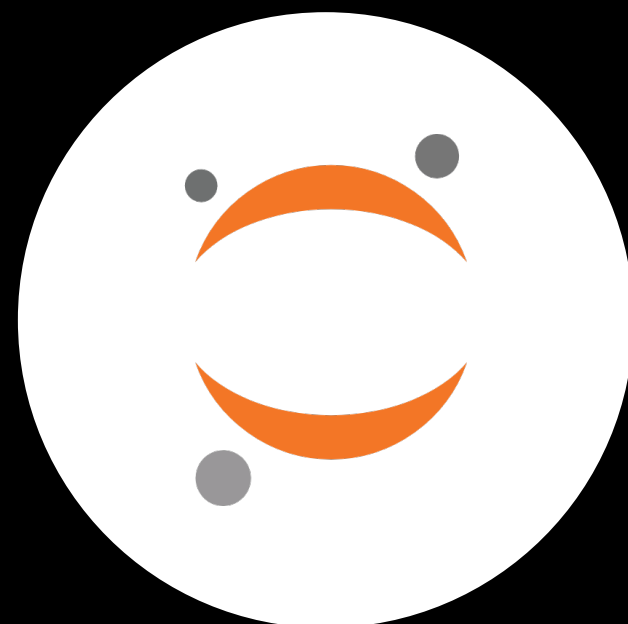
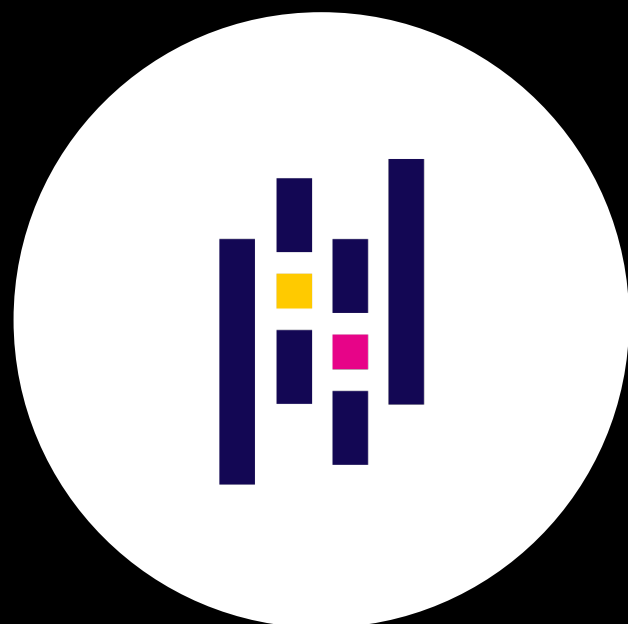


The Python's Scientific Ecosystem

Python 리터러시

01 Python 소개

1. Python 언어와
Python 생태계



2. Python을 이용한 개발 트렌드와 다양한 사용 사례



2023 Developer Survey

Python 리터러시

01 Python 소개

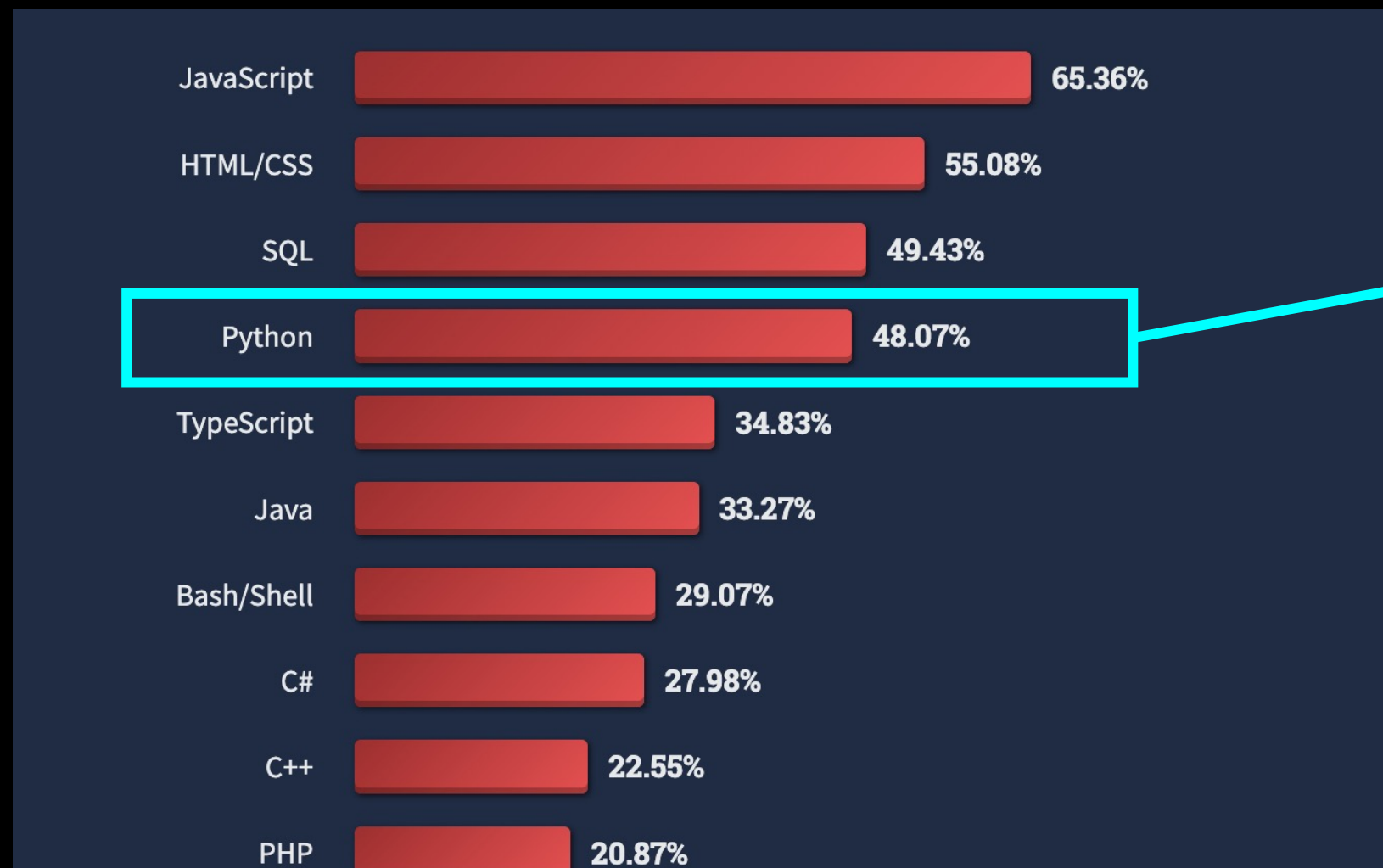
2. Python을 이용한
개발 트렌드와
다양한 사용 사례



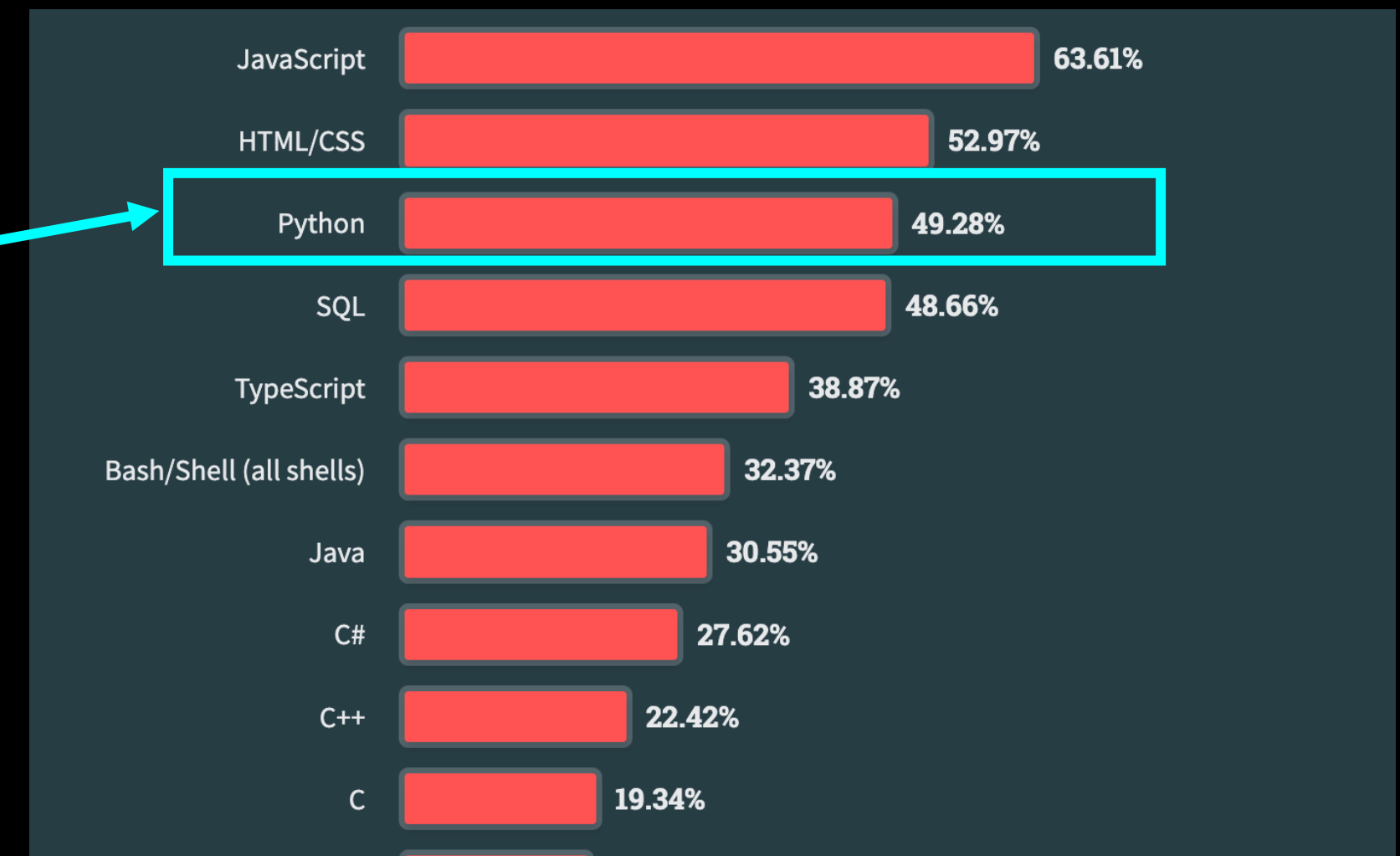
지난 1년(2023년) 동안 어떤 프로그래밍, 스크립트, 마크업 언어로 광범위한 개발 작업을 수행했으며, 내년에는 어떤 언어로 작업하고 싶나요?

Which programming, scripting, and markup languages have you done extensive development work in over the past year, and which do you want to work in over the next year?

2022



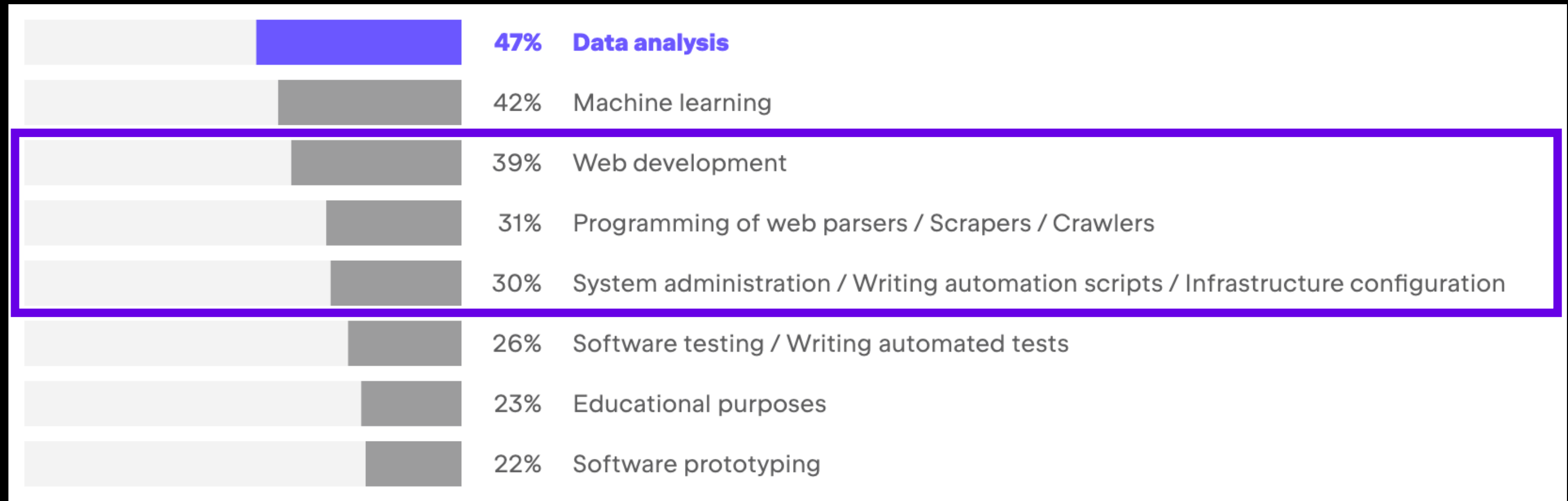
2023





Python을 어떤 용도로 사용하시나요?

What do you use Python for?



단 두 줄로 웹 애플리케이션 만들기

Python 리터러시

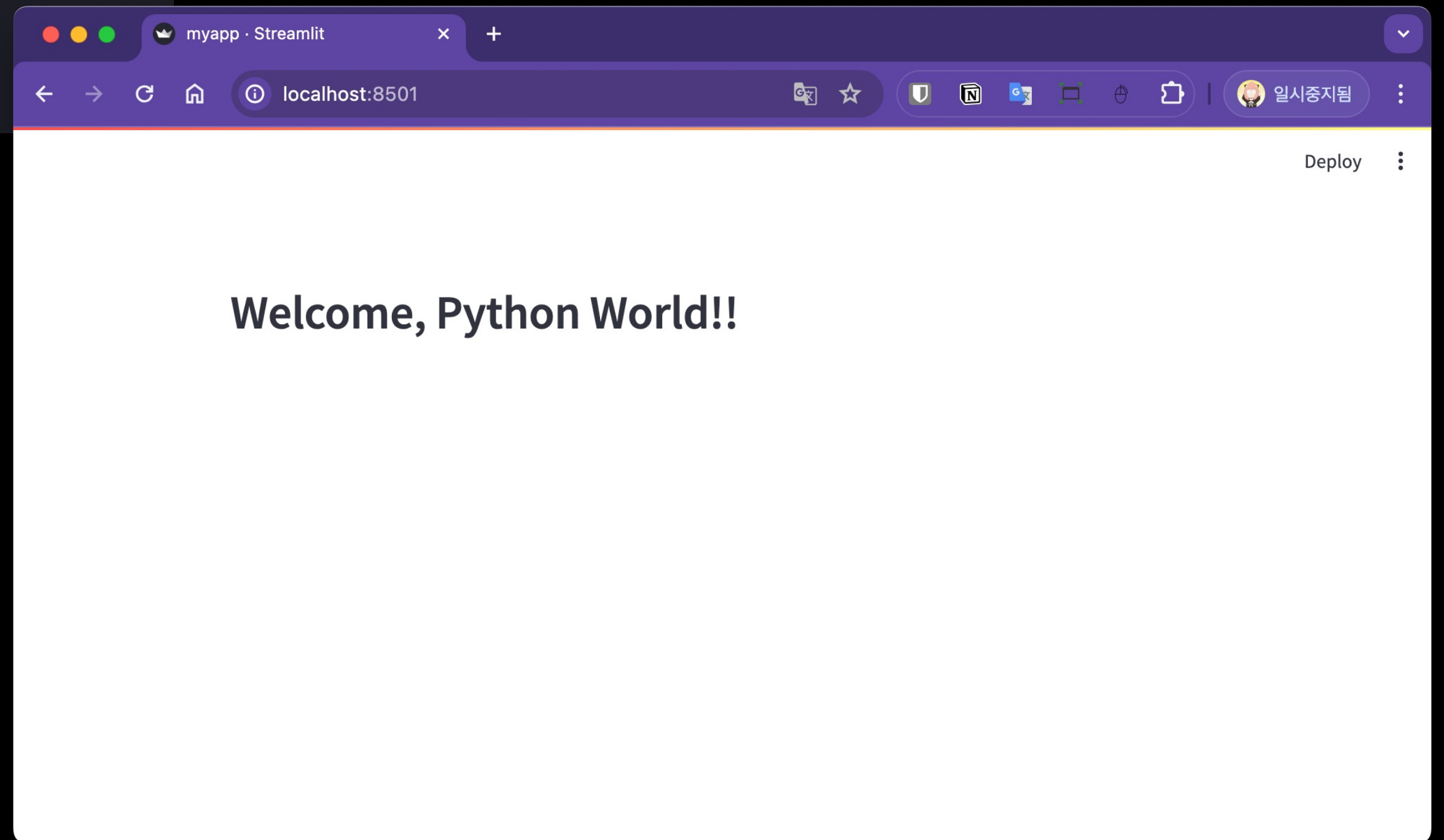
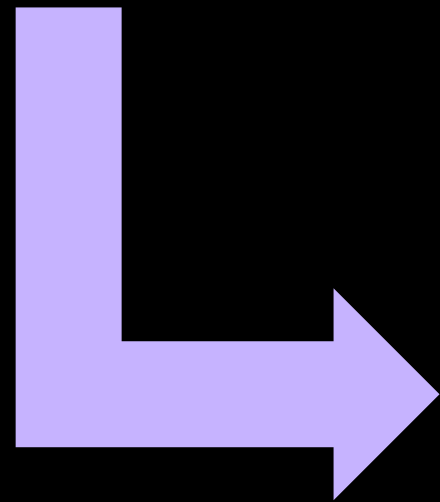
01 Python 소개

2. Python을 이용한
개발 트렌드와
다양한 사용 사례



```
import streamlit as st

st.header("Welcome, Python World!!")
```



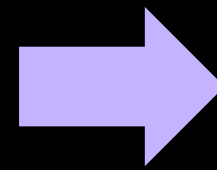
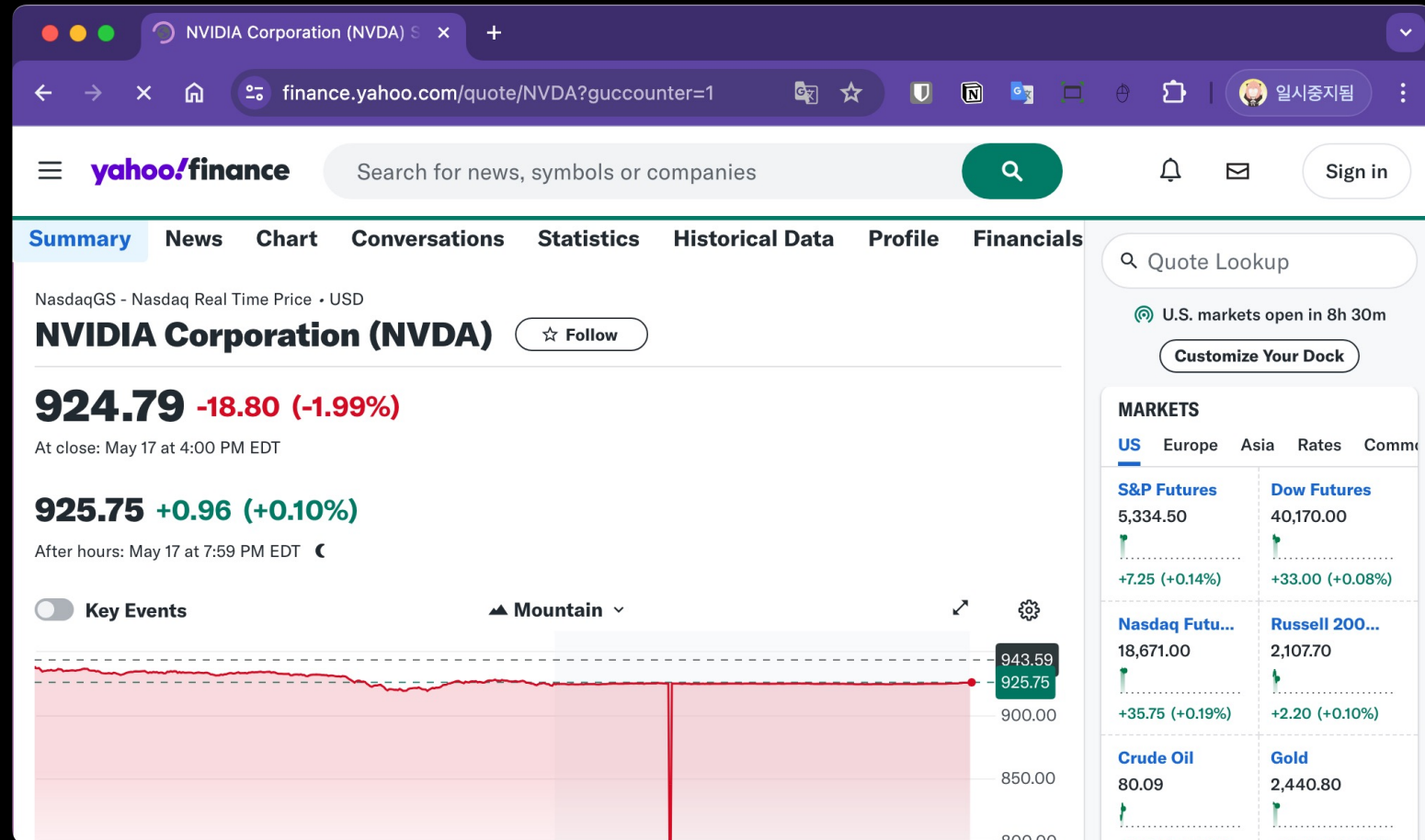


웹 스크래핑 과정 모식도

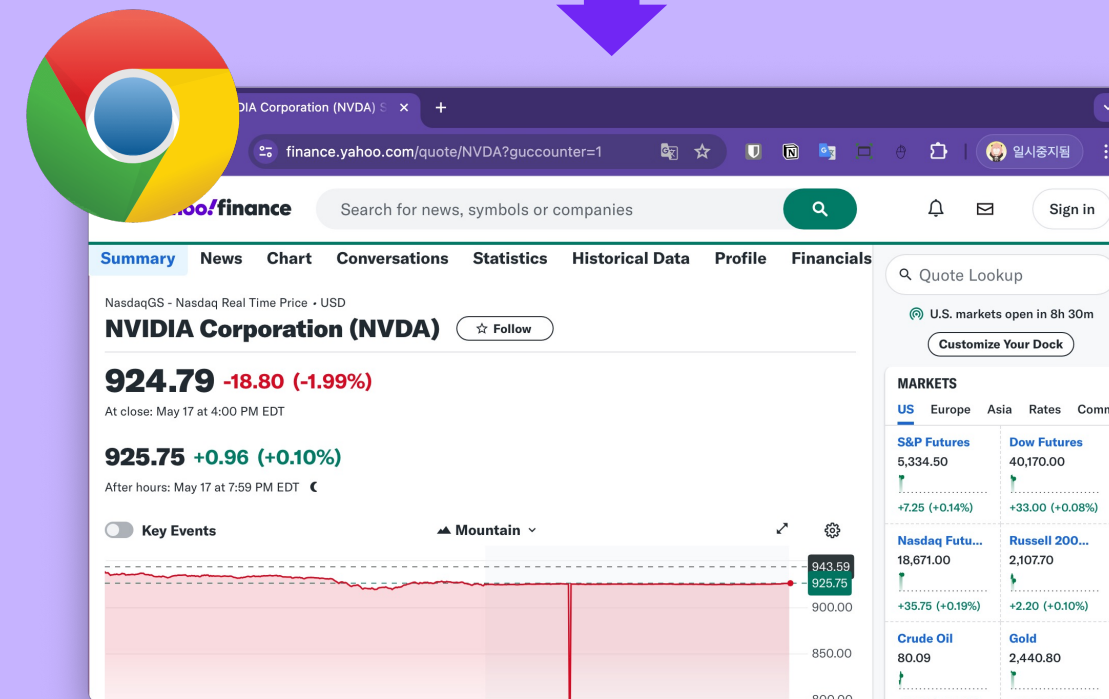
Python 리터러시

01 Python 소개

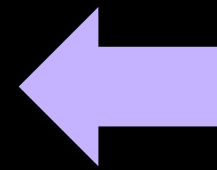
2. Python을 이용한
개발 트렌드와
다양한 사용 사례



웹 정보 요청



내용 추출



```
<div class="container svelte-mgkamr">
  <fin-streamer class="livePrice svelte-mgkamr" data-symbol="NVDA" data-testid="qsp-price" data-field="regularMarketPrice" data-trend="none" data-pricehint="2" data-value="924.79" active="">
    <span>924.79</span>
  </fin-streamer>

  <!-- Skip -->

</div>
```

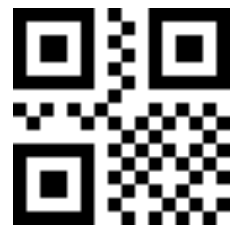


Python을 이용한 업무자동화

Python 리터러시

01 Python 소개

2. Python을 이용한
개발 트렌드와
다양한 사용 사례

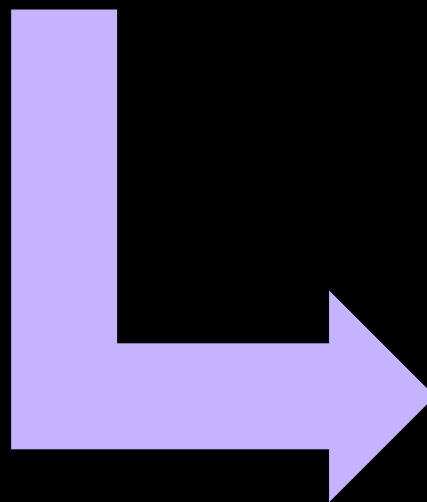


```
import qrcode
```

```
url_link = "https://www.python.org"
```

```
qr_img = qrcode.make(url_link)
```

```
qr_img.save("qr_img.png")
```



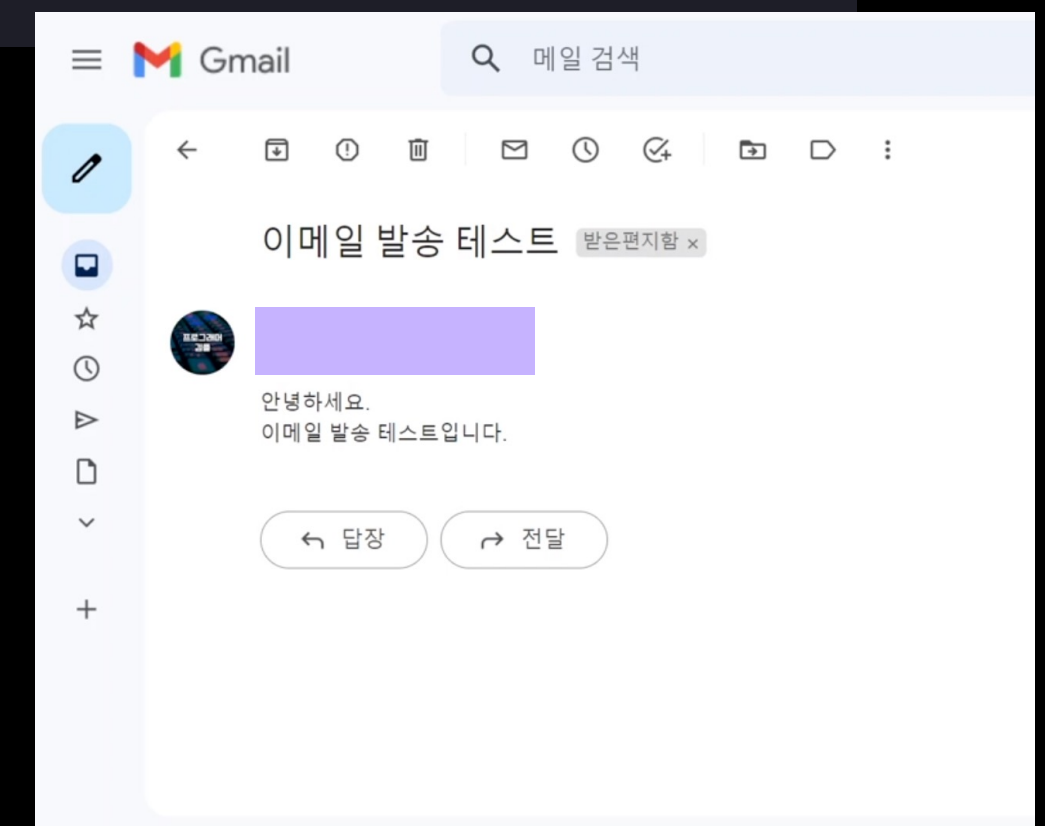
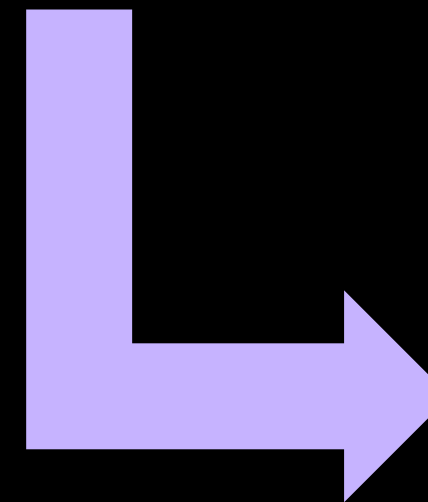
```
import smtplib  
from email.mime.text import MIMEText
```

```
sender_email = "k*****@elicer.com"  
sender_password = "*****"
```

```
receiver_email = "test@elicer.com"
```

```
email_title = "이메일 발송 테스트"  
email_message = "안녕하세요. \n이메일 발송 테스트입니다."
```

```
msg = MIMEText(email_message)  
msg['Subject'] = email_title  
msg['From'] = sender_email
```





Python의 인기가 높아지고 다양한 개발이 가능해지면서
다양한 분야 & 기업에서 사용 사례가 많아지고 있음

사례1. 자사 제품에 특화된 Python 개량판 배포

사례2. 자사 제품에 특화된 계산 성능을 가진 Python 기반의 프로그램(오픈소스)배포

사례3. 자사에서 보유한 데이터와 Python을 이용한 거대언어모델(LLM) 개발

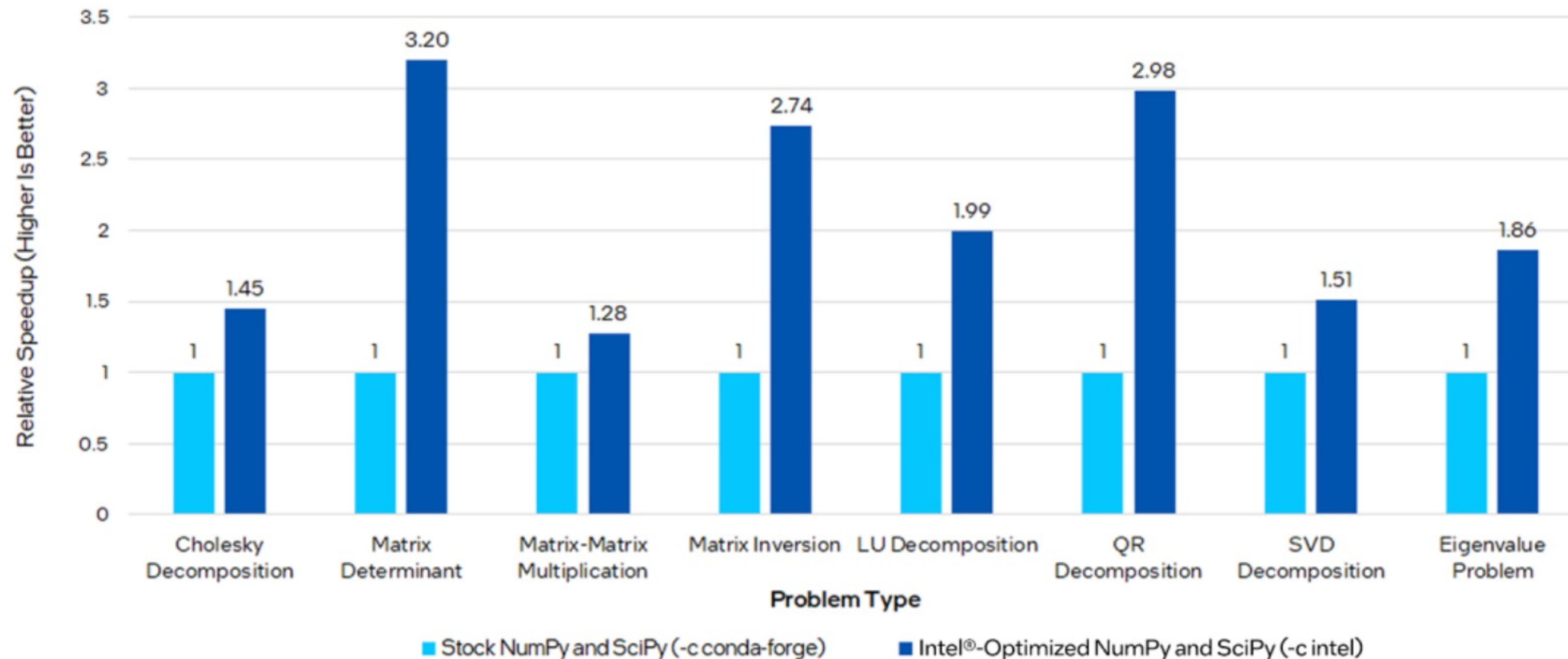


Intel 에서 제작하였으며 Intel 하드웨어에서 더 높은 성능을 발휘할 수 있도록 최적화된 Python Intel Math Kernel Library (Intel MKL)를 사용하여 수학 연산의 성능을 극대화하였고 다양한 연산(벡터 계산, 선형 대수 등)에 최적화



Intel®-Optimized NumPy* & SciPy* Linear Algebra Performance

Performance is increased up to 3.2x with Intel optimizations



Intel Optimization for NumPy and SciPy compared to conda-forge channel NumPy and SciPy performance for linear algebra on Intel® Xeon® Platform 8480+

Testing Date: Performance results are based on testing by Intel as of July 15, 2023, and may not reflect all publicly available updates.

Configuration Details and Workload Setup: System: cloud.intel.com, nodes=1; spr; ppn=2, Intel® Xeon® Platinum 8480+, 2 sockets, 56 cores per socket, HT On, Intel® Turbo Boost Technology on, Total memory 528 GB, RAM 33 MHz, Ubuntu® 20.04.5 LTS, 5.18.15-051815-generic, Microcode: 0x2b000310, benchmarks <https://github.com/IntelPython/ibench-Linear-Algebra>, -c conda-forge environment versions: NumPy 1.23.5, SciPy 1.10.1, Numba* 0.56.4 modules installed, -c intel environment versions: NumPy 1.21.4, SciPy 1.7.3, Numba* 0.56.3, tbb4py 2021.8.0 modules installed.

Performance varies by use, configuration and other factors. See backup for configuration, details.



NVIDIA의 CUDA 기술을 활용해 Python에서 GPU 가속 연산을 수행할 수 있게 해주는 오픈소스 프로그램
Python 언어의 간결함과 편리함을 유지하면서, 대규모 데이터 처리와 복잡한 수치 계산 작업을 위한 강력한 GPU 가속 활용 가능

```
[1038943] python3.9@127.0.0.1
saxpy, 2021-Apr-02 08:42:41, Context 2, Stream 20
Section: GPU Speed Of Light
-----
DRAM Frequency                cycle/nsecond          6.47
SM Frequency                   cycle/nsecond          1.36
Elapsed Cycles                 cycle                  474,147
Memory [%]                    %                      92.84
SOL DRAM                      %                      92.84
Duration                      usecond               348.93
SOL L1/TEX Cache              %                      15.84
SOL L2 Cache                  %                      29.26
SM Active Cycles              cycle                 459,277.39
SM [%]                        %                      12.30
-----
OK   The kernel is utilizing greater than 80.0% of the available compute or memory performance of the device. To
      further improve performance, work will likely need to be shifted from the most utilized to another unit.
      Start by analyzing workloads in the Memory Workload Analysis section.

Section: Launch Statistics
-----
Block Size                    512
Function Cache Configuration  cudaFuncCachePreferNone
Grid Size                     32,768
Registers Per Thread          register/thread         16
Shared Memory Configuration Size Kbyte                 32.77
Driver Shared Memory Per Block byte/block                0
Dynamic Shared Memory Per Block byte/block                0
Static Shared Memory Per Block byte/block                0
Threads                      thread                 16,777,216
Waves Per SM                 227.56
-----

Section: Occupancy
-----
Block Limit SM                block                  16
Block Limit Registers          block                   8
Block Limit Shared Mem         block                  16
Block Limit Warps              block                   2
Theoretical Active Warps per SM warp                      32
Theoretical Occupancy          %                      100
Achieved Occupancy             %                      87.63
Achieved Active Warps Per SM   warp                   28.04
-----
WRN   This kernel's theoretical occupancy is not impacted by any block limit. The difference between calculated
      theoretical and measured achieved occupancy can be the result of warp scheduling overheads or workload
      imbalances during the kernel execution. Load imbalances can occur between warps within a block as well as
      across blocks of the same kernel.
```

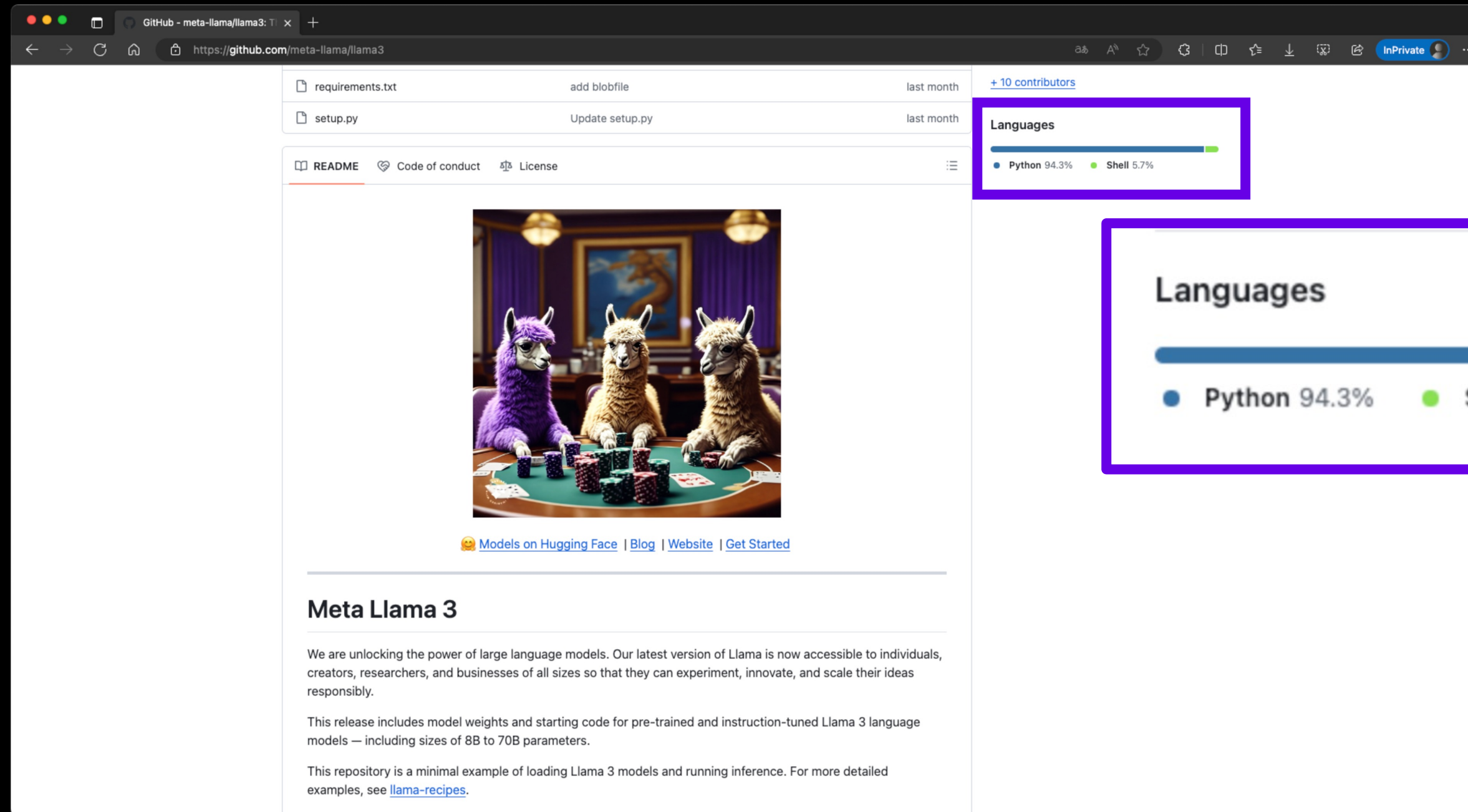
	C++	Python
Kernel execution	352μs	352μs
Application execution	1076ms	1080ms

Meta : Open source LLM – Llama

Python 리터러시

01 Python 소개

2. Python을 이용한
개발 트렌드와
다양한 사용 사례



Meta(전 facebook)에서 제작한 오픈소스 거대언어모델(LLM)인 LLaMA3 는
Python 기반의 Pytorch 로 실행 시킬 수 있음

사용자는 자체적으로 LLM을 제작하거나 튜닝할 수 있음

Meta : Open source LLM – Llama

Python 리터러시

01 Python 소개

2. Python을 이용한
개발 트렌드와
다양한 사용 사례

	Llama-3-8B	Gemma 7B - It	Mistral 7B Instruct
MMLU	68.4	53.3	58.4
GPQA	34.2	21.4	26.3
HUMAN EVAL	62.2	30.5	36.6
GSM-8K	79.6	30.6	39.9
MATH	30.0	12.2	11.0

	Llama-3-70B	Gemini Pro 1.5	Claude3 Sonnet
MMLU	82.0	81.9	79.0
GPQA	39.5	41.5	38.5
HUMAN EVAL	81.7	71.9	73.0
GSM-8K	93.0	91.7	92.3
MATH	50.4	58.5	40.5

새로 출시된 Llama3는 다양한 모델과 비교했을 때, 준수한 성능을 보임